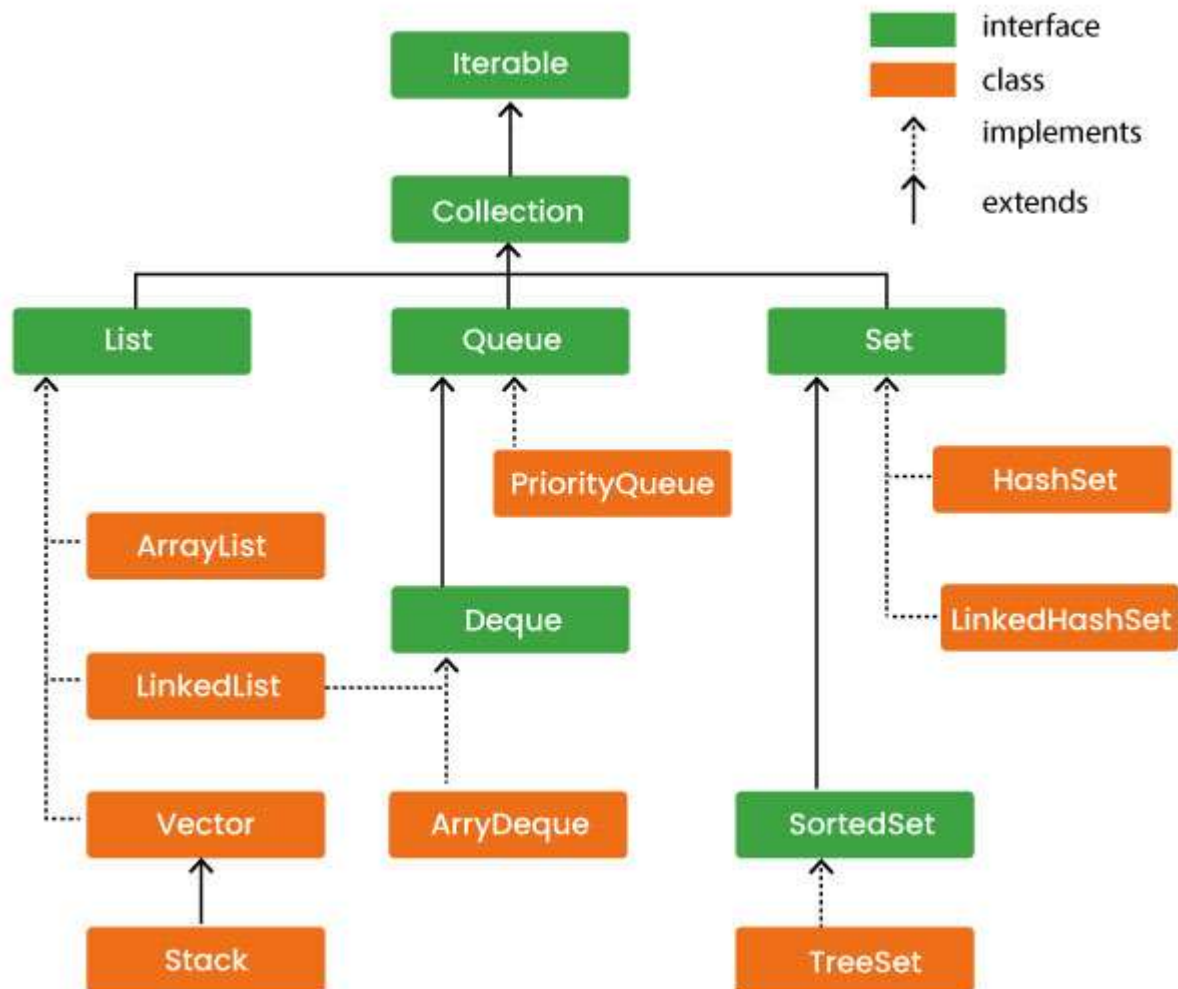


Collection Framework



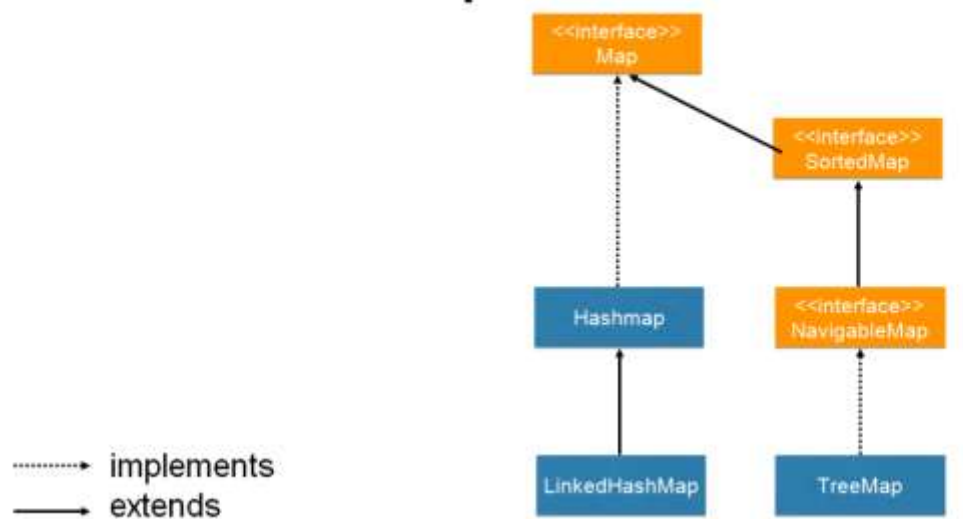
What is Collection Framework?

The **Java Collection Framework (JCF)** is a set of classes and interfaces used to store, manage, and manipulate groups of objects.

Key Points

- Helps in **data storage & retrieval**
- Provides **ready-made data structures**
- Part of java.util package
- Reduces coding effort

Map Interface



Why Use Collection Framework?

- No need to implement data structures manually
- Provides **dynamic memory allocation**
- Built-in methods like:
 - add()
 - remove()
 - sort()
- Improves **performance & reusability**

Main Interfaces

Interface	Description
List	Ordered collection, allows duplicates
Set	Unordered, no duplicates
Queue	FIFO (First In First Out)
Map	Key-Value pair (no duplicate keys)

Collection interface methods

Category	Method	Return Type	Description
Add Operations	add(E e)	boolean	Adds a single element
	addAll(Collection<? extends E> c)	boolean	Adds all elements from another collection
Remove Operations	remove(Object o)	boolean	Removes a specific element
	removeAll(Collection<?> c)	boolean	Removes all matching elements
	retainAll(Collection<?> c)	boolean	Keeps only common elements
	clear()	void	Removes all elements
Search Operations	contains(Object o)	boolean	Checks if element exists
	containsAll(Collection<?> c)	boolean	Checks if all elements exist
Size / State	size()	int	Returns number of elements
	isEmpty()	boolean	Checks if collection is empty
Traversal	iterator()	Iterator	Returns iterator for traversal
Conversion	toArray()	Object[]	Converts to array
Comparison	equals(Object o)	boolean	Compares two collections
	hashCode()	int	Returns hash code

ArrayList

- **ArrayList** is an **implemented class of List interface**
- It is present in **java.util package**
- ArrayList is created on the basis of:
Growable / Resizable Array

ArrayList is a growable array-based implementation of List interface that stores elements in insertion order and allows duplicates.

Syntax

```
package java.util;  
class ArrayList implements List  
{  
    // constructors  
    // methods  
}
```

Properties of ArrayList

1. ArrayList is an **index-based data structure**
2. ArrayList can store **different data types** (*heterogeneous data*)
3. ArrayList can store **duplicate values**
4. ArrayList can store **any number of null values**
5. ArrayList **follows insertion order**
6. ArrayList **does not follow sorting order**

ArrayList Integer Object Type :

2	5	12	1	79	11
0	1	2	3	4	5

Integer type (for all indices) Data

Traversing

We can traverse

1. By using looping statement
2. By using for each loop
3. By using Iterator
4. By using ListIterator

```
import java.util.ArrayList;

public class Traverse {
    public static void main(String[] args) {

        ArrayList<Integer> list = new ArrayList<>();
        list.add(10);
        list.add(20);
        list.add(30);

        //! by using looping

        for(int i=0 ; i<list.size(); i++)
        {
            System.out.println(list.get(i));
        }

        // ! by using for each loop

        for(int ele : list)
        {
            System.out.println(ele);
        }

    }
}
```

Iterator

- Used to **traverse (iterate) elements one by one**
- Works on **all Collection classes**

Methods of Iterator

Method	Description
hasNext()	Checks if next element is present
next()	Returns next element
remove()	Removes current element

Example

```
import java.util.*;

public class Demo {
    public static void main(String[] args) {

        ArrayList<Integer> list = new ArrayList<>();
        list.add(10);
        list.add(20);
        list.add(30);

        Iterator<Integer> it = list.iterator();

        while (it.hasNext()) {
            int val = it.next();
            System.out.println(val);

            if (val == 20) {
                it.remove();
            }
        }

        System.out.println("After removal: " + list);
    }
}
```

ListIterator

- Works only on **List (ArrayList, LinkedList)**
- Can traverse **both directions**

Methods

Method	Description
hasNext()	Forward check
next()	Move forward
hasPrevious()	Backward check
previous()	Move backward
add(E e)	Add element
set(E e)	Replace element
remove()	Remove element

Example

```
import java.util.ArrayList;
import java.util.ListIterator;

public class Demo {
    public static void main(String[] args) {

        ArrayList<Integer> list = new ArrayList<>();
        list.add(10);
        list.add(20);
        list.add(30);

        ListIterator<Integer> li = list.listIterator();
```

```

        // Forward

        while (li.hasNext()) {
            int val = li.next();
            System.out.println(val);
            if (val == 20) {
                li.set(25); // replace
            }
        }

        System.out.println("after replace : "+list);

        System.out.println("-----backward-----");

        // Backward
        while (li.hasPrevious()) {
            int ele = li.previous();
            System.out.println(ele);
            if(ele == 30)
                li.remove(); // remove

        }

        System.out.println("after remove : "+list);

        ArrayList<Integer> list2 = new ArrayList<>(Arrays.asList(10, 20, 30));

        ListIterator<Integer> li2 = list.listIterator();

        // add

        while (li2.hasNext()) {
            int val = li2.next();

            if (val == 20) {
                li2.add(25);
            }
        }

        System.out.println(list2);

    }
}

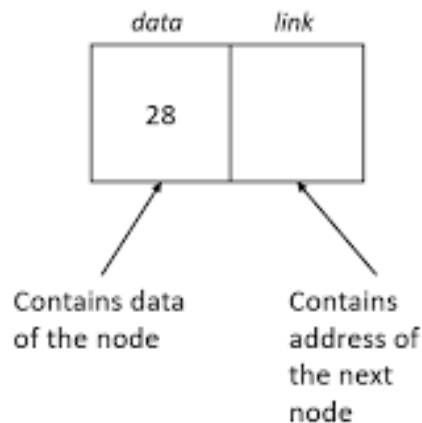
```

Iterator vs ListIterator

Feature	Iterator	ListIterator
Works on	All collections	Only List
Direction	Forward only	Forward + Backward
Add element	Not allowed	Allowed
Replace (set)	Not allowed	Allowed
Remove	Allowed	Allowed
Traverse backward	No	Yes

LinkedList

1. A **LinkedList** is a linear data structure where elements are stored as **nodes**, and each node contains:
 - **Data**
 - **Reference (address)** to the next node



2. LinkedList is an implemented class of List interface.
3. LinkedList is a class present in **java.util** package

Note :

Unlike ArrayList, elements are **not stored in contiguous memory**.

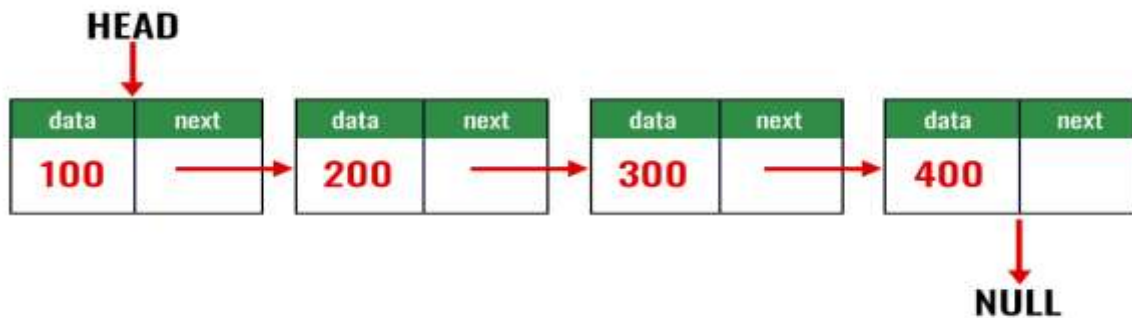
How to declare :

```
LinkedList<Integer> list = new LinkedList<>();
```

Types of Linked List

1. Singly Linked List

- Each node points to next node only

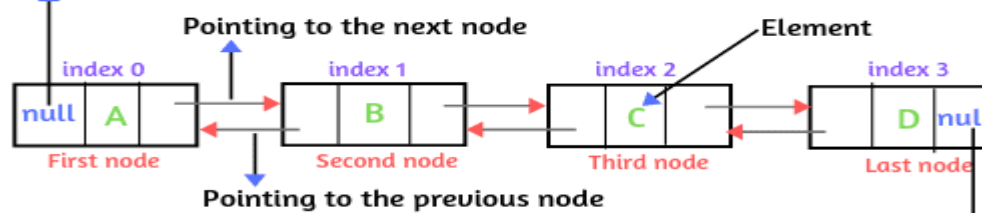


2. Doubly Linked List

- Each node has:
 - previous pointer
 - next pointer



Here, null indicates that there is no previous element.

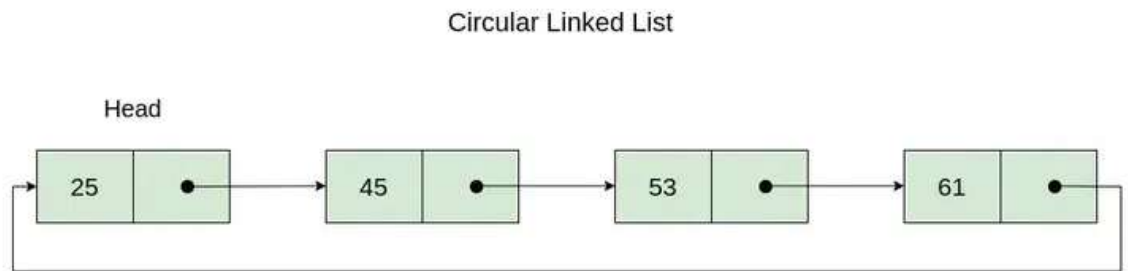


Here, null indicates that there is no next element.

A array representation of linear Doubly LinkedList in Java

3. Circular Linked List

- Last node points to first node



Note: Java LinkedList is **Doubly Linked List**

Advantages

- Insertion and deletion operation is efficient and fast.
- We can traverse in forward and backward.
- No need to define size initially

Disadvantages

- Traversal is required to access elements
- More memory usage , in case of insertion and deletion we have to manage previous and next node.

Properties

1. Index-Based Data Structure

- LinkedList supports **index-based access**
- We can access elements using:

Example :

```
list.get(index);
```

2. Allows Heterogeneous Data (Conceptual)

- Can store **different types of elements**
- But in Java, we usually use **generics** for type safety

Example:

```
list.add(10);  
list.add("Hello");  
list.add(5.5);
```

3. Allows Duplicate Elements

- Duplicate values are allowed

Example:

```
list.add(10);  
list.add(10);
```

4. Allows Multiple Null Values

- LinkedList can store **null values**

Example:

```
list.add(null);  
list.add(null);
```

5. Maintains Insertion Order

- Elements are stored in the **order they are inserted**

Example:

10 → 20 → 30 → 40

6. Does Not Follow Sorting Order

- LinkedList does **not sort automatically**
- Order depends only on insertion

LinkedList vs ArrayList

Feature	ArrayList	LinkedList
Nature	Acts as List only	Acts as List + Deque
Underlying Data Structure	Resizable Array (Dynamic Array)	Doubly Linked List
Memory Storage	Elements stored in contiguous memory locations	Elements stored in non-contiguous memory locations
Insertion & Deletion	Slow (shifting required)	Fast (no shifting, only pointer change)
Retrieval (Access)	Fast (index-based access)	Slow (needs traversal)

When to Use LinkedList?

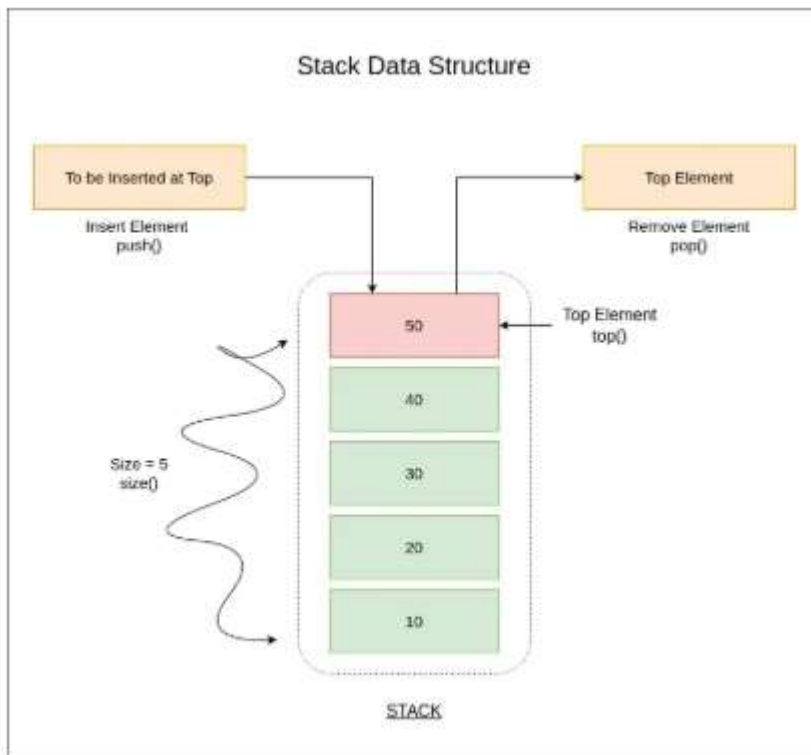
Use LinkedList when:

- Frequent insertion/deletion
- No need for fast random access

Avoid when:

- we need fast indexing (get(i))

Stack



Stack is a linear data structure that follows LIFO principle where insertion and deletion happen only at one end called TOP.

- **Stack** is a **legacy class** introduced in **JDK 1.0**
- It is part of **java.util package**
- Stack **extends Vector class**

Syntax

```
import java.util.Stack;
```

```
Stack<Type> stack = new Stack<>();
```

Stack Principle

Stack follows:

LIFO (Last In First Out)

The **last inserted element** is removed first

Properties :

1. Stack is **index-based data structure**
2. Elements are inserted and removed only from **TOP**
3. Stack can be implemented using:
 - Array
 - ArrayList
 - LinkedList

Method	Return Type	Description	Example
push(E item)	E	Adds element to top of stack	stack.push(10);
pop()	E	Removes and returns top element	stack.pop();
peek()	E	Returns top element without removing	stack.peek();
empty()	boolean	Checks if stack is empty	stack.empty();
search(Object o)	int	Returns position from top (1-based index)	stack.search(10);

Valid Parentheses (leetcode 20)

Example 1:

Input: s = "()"

Output: true

Example 2:

Input: s = "()[]{}"

Output: true

Example 3:

Input: s = "("

Output: false

Example 4:

Input: s = "([)]"

Output: true

```
import java.util.*;

class Solution {
    public boolean isValid(String s) {
        Stack<Character> stack = new Stack<>();

        char arr[] = s.toCharArray();

        for(char ch : arr)
        {
            if(ch=='(' || ch=='{' || ch=='[')
                stack.push(ch);
            else
            {
                if(stack.empty()) return false;

                char top = stack.pop();

                if(ch==')' && top!='(') return false;
                else if(ch=='}' && top!='{') return false;
                else if(ch==']' && top!='[') return false;
            }
        }

        return stack.empty();
    }
}
```

Remove all adjacent duplicates in String (1047)

Example 1:

Input: s = "abbaca"

Output: "ca"

Explanation:

For example, in "abbaca" we could remove "bb" since the letters are adjacent and equal, and this is the only possible move. The result of this move is that the string is "aaca", of which only "aa" is possible, so the final string is "ca".

Example 2:

Input: s = "azxxzy"

Output: "ay"

```
class Solution {
    public String removeDuplicates(String s) {

        Stack<Character> st = new Stack<>();

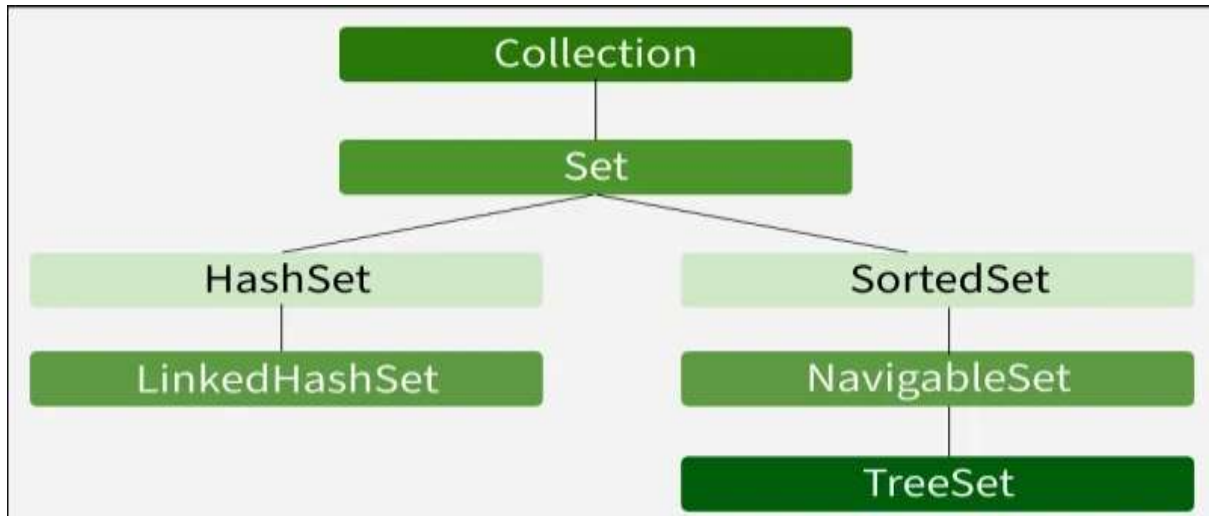
        for(char ch : s.toCharArray())
        {
            if(!st.isEmpty() && st.peek()==ch)
            {
                st.pop();
            }
            else
                st.push(ch);
        }

        StringBuilder s2 = new StringBuilder();

        for(char ch : st)
        {
            s2.append(ch);
        }

        return s2.toString();
    }
}
```

Set



Feature	List	Set
Data Structure Type	Index based data structure	Not index based data structure
Storage Logic	Stores elements based on index position	Stores elements based on hashcode values(except LinkedHashset)
Duplicate Elements	Allows duplicate elements	Does not allow duplicate elements
Null Values	Can store multiple null values	Can store only one null value(except TreeSet)
Order	Follows insertion order	Does not follow insertion order(except TreeSet)
Iteration	Can use Iterator and ListIterator	Can use Iterator only

HashSet

- **HashSet** is an implementation class of the **Set** interface
- It is present in the **java.util** package
- The underlying data structure of **HashSet** is:
HashTable
- HashTable initial capacity is 16.
- HashSet was introduced in **JDK 1.2**

Syntax

```
import java.util.HashSet;

class Example {
    public static void main(String[] args) {
        HashSet<Integer> set = new HashSet<>();
    }
}
```

Properties:

1. **Not index-based**
 - Elements are stored using **hashing (hashcode)**
2. **No duplicate elements allowed**
 - Duplicate values are automatically ignored
3. **Allows only one null value**
 - Multiple nulls are not allowed
4. **Heterogeneous data allowed**
 - Can store different types of objects
5. **No insertion order maintained**
 - Order is not guaranteed
6. **No sorting**
 - Elements are not sorted

Methods

Method	Description	Example
add(E e)	Adds an element to the set	set.add(10);
clear()	Removes all elements	set.clear();
contains(Object o)	Checks if element exists	set.contains(10);
isEmpty()	Checks if set is empty	set.isEmpty();
size()	Returns number of elements	set.size();
iterator()	Returns iterator to traverse	Iterator it = set.iterator();
toArray()	Converts set to array	set.toArray();
equals(Object o)	Compares two sets	set1.equals(set2);

1. Remove Duplicate from array

```
import java.util.*;

public class RemoveDuplicate {
    public static void main(String[] args) {
        int arr[] = {1, 2, 2, 3, 4, 4, 5};

        HashSet<Integer> set = new HashSet<>();

        for (int num : arr) {
            set.add(num);
        }

        System.out.println("Unique elements: " + set);
    }
}
```

2. Check Duplicate

```
public class CheckDuplicate {  
    public static boolean duplicatePresent(int[] arr)  
    {  
        HashSet<Integer> set = new HashSet<>();  
        for (int num : arr) {  
            if (set.contains(num)) {  
                return true;  
            } else {  
                set.add(num);  
            }  
        }  
        return false;  
    }  
}
```

3.Union Of two Arrays

```
public class UnionArray {
    public static void main(String[] args) {
        int a[] = {1, 2, 3};
        int b[] = {3, 4, 5};

        HashSet<Integer> set = new HashSet<>();

        for (int x : a)
            set.add(x);
        for (int x : b)
            set.add(x);

        System.out.println("Union: " + set);
    }
}
```

4.Intersection of two arrays

```
public class IntersectionArray {
    public static void main(String[] args) {
        int a[] = {1, 2, 3, 4};
        int b[] = {3, 4, 5, 6};

        HashSet<Integer> set = new HashSet<>();
        HashSet<Integer> result = new HashSet<>();

        for (int x : a) set.add(x);

        for (int x : b) {
            if (set.contains(x)) {
                result.add(x);
            }
        }

        System.out.println("Intersection: " + result);
    }
}
```

LinkedHashSet

- **LinkedHashSet** is an implementation class of the **Set** interface
- It is present in the **java.util** package
- It is a combination of:
 HashSet + LinkedList
- Uses:
 HashTable (for storage)
 Doubly Linked List (for order)

Syntax

```
import java.util.LinkedHashSet;

class Example {
    public static void main(String[] args) {
        LinkedHashSet<Integer> set = new LinkedHashSet<>();
    }
}
```

Properties of LinkedHashSet

1. **Maintains insertion order**
 - Elements are stored in the order they are inserted
2. **No duplicate elements**
 - Duplicate values are not allowed
3. **Allows one null value**
 - Only one null is allowed
4. **Not index-based**
 - No indexing like array
5. **Heterogeneous data allowed**
 - Can store different types

TreeSet

TreeSet is a direct implementation class of **NavigableSet** Indirectly implements:

- **SortedSet**
- **Set**
- TreeSet was introduced in **JDK 1.2**
- Underlying data structure:
Balanced Tree (Red-Black Tree)
-

Syntax

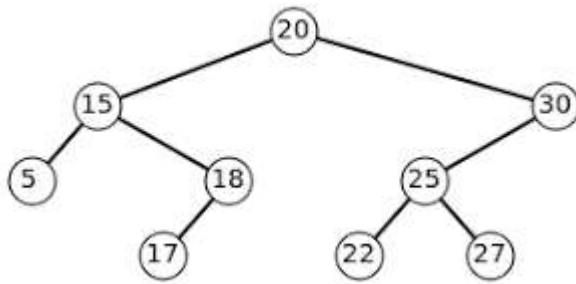
```
import java.util.TreeSet;

class Example {
    public static void main(String[] args) {
        TreeSet<Integer> set = new TreeSet<>();
    }
}
```

Properties of TreeSet

1. **Not index-based**
 - Elements are not stored using index
2. **Does NOT follow insertion order**
 - Order depends on sorting
3. **Follows sorting order**
 - Default: **ascending order**
4. **Stores homogeneous data**
 - All elements must be of the **same type**
5. **Does NOT allow null**
 - Inserting null → Runtime error
6. **Does NOT allow duplicates**
 - Duplicate elements are ignored

Example Representation (Sorting)



Example Program

```
import java.util.*;

public class Demo {
    public static void main(String[] args) {

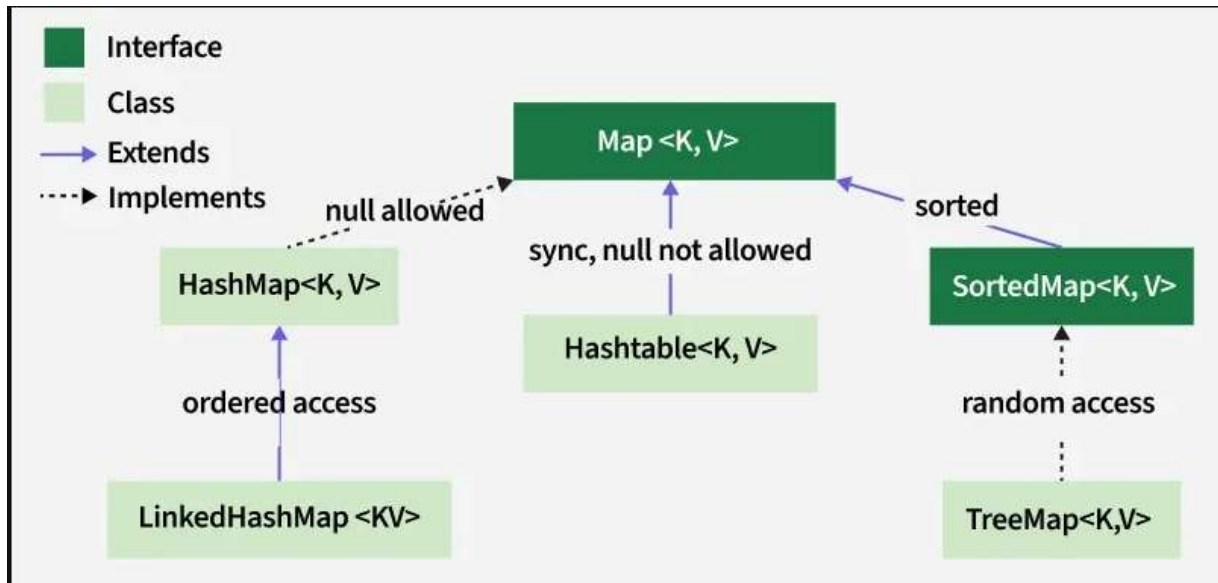
        TreeSet<Integer> set = new TreeSet<>();

        set.add(40);
        set.add(20);
        set.add(60);
        set.add(10);
        set.add(30);

        System.out.println(set);
    }
}

// Output
// [10, 20, 30, 40, 60]
```

Map



Map is a interface that is present in **java.util** package.

Map does not inherit Collection interface.

Map was invented in JDK 1.2 version.

Properties

1. Key-Value Pair

- A Map stores data in key-value pairs.
- Each pair is called an entry.

Example:

Key → Value
101 → Deepak
102 → Amit
103 → Rahul
104 → Deepak

2. Keys Must Be Unique

- Keys cannot be duplicated.
- Values can be duplicated.

Valid:

101 → Deepak

102 → Deepak

Invalid:

101 → Amit

101 → Rahul (duplicate key not allowed)

3. Null Values

- Map allows only one null key (in HashMap).
- Map allows multiple null values.

Example:

null → A

101 → null

102 → null

4. Ordering

- Map does not follow insertion order (in HashMap).
- It does not sort automatically.
- Output order is not guaranteed.

Methods of Map

Method	Description	Example
put(K key, V value)	Adds or updates a key-value pair	map.put(101, "Amit")
get(Object key)	Returns value for the given key	map.get(101)
remove(Object key)	Removes mapping for a key	map.remove(101)
containsKey(Object key)	Checks if key exists	map.containsKey(101)
containsValue(Object value)	Checks if value exists	map.containsValue("Amit")
size()	Returns number of entries	map.size()
isEmpty()	Checks if map is empty	map.isEmpty()
clear()	Removes all entries	map.clear()
keySet()	Returns all keys as Set	map.keySet()
values()	Returns all values as Collection	map.values()
entrySet()	Returns all key-value pairs	map.entrySet()
replace(K key, V value)	Replaces value for a key	map.replace(101, "Deepak")
getOrDefault(Object key, V defaultValue)	Returns value or default	map.getOrDefault(105, "Not Found")

HashMap

- HashMap is an implemented class of the Map interface.
- It is used to store data in key-value pairs.

Syntax

```
import java.util.*;
```

```
HashMap<Integer, String> map = new HashMap<>();
```

Properties of HashMap

1. Key-Value Pair

- HashMap stores data in key-value form.
- Each pair is called an entry.

2. Unique Keys

- Keys must be unique, If duplicate key is added, old value is replaced.

Example:

```
map.put(101, "Amit");  
map.put(101, "Rahul"); // replaces Amit
```

3. Duplicate Values Allowed :

Multiple keys can have the same value.

4. Null Values

- Allows only one null key.
- Allows multiple null values.

5. No Ordering

- Does not maintain insertion order.
- Output order is not guaranteed.

```
import java.util.HashMap;
import java.util.Map;

public class MapLearning {

    public static void main(String[] args) {

        // ! how to create
        HashMap<Integer,String> map = new HashMap<>();

        // ! how to add
        map.put(1, "rahul");
        map.put(2, "rohit");
        map.put(3, "virat");
        map.put(1, "sachin");
        map.put(4, "gautam");

        // ! how to print
        System.out.println(map);

        // ! how to delete
        map.remove(2);
        System.out.println(map);

        // ! how to know the size

        System.out.println(map.size());

        // ! how to get the value

        System.out.println(map.get(4));

        // ! how to check present or not

        System.out.println(map.containsKey(4));
        System.out.println(map.containsKey(2));

        System.out.println("key ----> value");

        for(Map.Entry<Integer,String> entry : map.entrySet())
        {
            System.out.println(entry.getKey() + "---->" + entry.getValue());
        }

    }
}
```

LinkedHashMap

- LinkedHashMap is a class that implements the Map interface.
- It stores data in key-value pairs.
- It **maintains the insertion order** of elements.

Syntax

```
import java.util.*;
```

```
LinkedHashMap<Integer, String> map = new LinkedHashMap<>();
```

When to Use LinkedHashMap

- When insertion order is important

Example Program

```
import java.util.*;

public class LinkedHashMapExample {
    public static void main(String[] args) {

        LinkedHashMap<Integer, String> map = new LinkedHashMap<>();

        map.put(101, "Amit");
        map.put(102, "Rahul");
        map.put(103, "Deepak");

        System.out.println(map);
    }
}

//output
{101=Amit, 102=Rahul, 103=Deepak}
```

TreeMap

- TreeMap is a class that implements the Map interface.
- It stores data in key-value pairs.
- It stores keys in **sorted order**.

Syntax

```
import java.util.*;
```

```
TreeMap<Integer, String> map = new TreeMap<>();
```

Properties of TreeMap

1. Sorted Keys

- Keys are stored in **ascending order by default**.
- Sorting is based on:
 - Natural ordering (for numbers, strings)
 - Or custom comparator

Example:

```
map.put(3, "C");  
map.put(1, "A");  
map.put(2, "B");  
  
System.out.println(map);  
// Output: {1=A, 2=B, 3=C}
```

2. Null Handling

- Does NOT allow null key.
- Allows multiple null values.

When to Use TreeMap

- When sorted data is required

HashTable

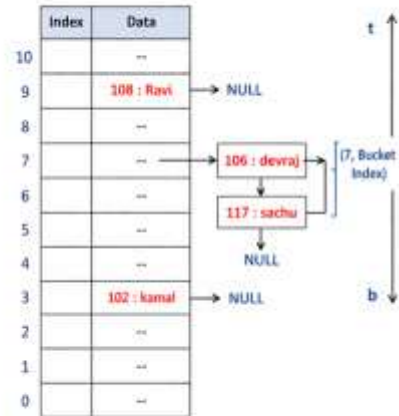
Hashtable:-

- ① **Hashtable** is the direct implemented class of Map interface.
- ② **Syntax:-**

```
package java.util;  
class Hashtable implements Map  
{  
    // constructors  
    // methods  
}
```
- ③ It is a legacy class & was introduced in **JDK 1.0** version.
- ④ **Hashtable** underlying DS is **hashtable**.

> Working of Hashtable

- `Hashtable ht = new Hashtable();`
- `ht.put(106, "devraj");`
 $106 \% 11 = 7$
- `ht.put(102, "kamal");`
 $102 \% 11 = 3$
- `ht.put(108, "Ravi");`
 $108 \% 11 = 9$
- `ht.put(117, "sachu");`
 $117 \% 11 = 7$



• 108:Ravi, 117:sachu, 106:devraj, 102:kamal

Array + HashMap Important Questions (LeetCode)

Easy Level

1. Two Sum (#1)
2. Contains Duplicate (#217)
3. First Unique Character in a String (#387)
4. Intersection of Two Arrays (#349)

Medium Level

5. Subarray Sum Equals K (#560)
6. Longest Consecutive Sequence (#128)
7. Top K Frequent Elements (#347)
8. Group Anagrams (#49)

High Level

9. Majority Element (#169)
10. Find All Anagrams in a String (#438)
11. Continuous Subarray Sum (#523)

169. Majority Element

Easy Topics Compare

Given an array `nums` of size `n`, return the *majority element*.

The majority element is the element that appears more than $\lfloor n / 2 \rfloor$ times. You may assume that the majority element always exists in the array.

Example 1:

Input: `nums = [3,2,3]`
Output: `3`

Example 2:

Input: `nums = [2,2,1,1,1,2,2]`
Output: `2`

```
import java.util.HashMap;

class Solution {
    public int majorityElement(int[] arr) {

        int n = arr.length;

        HashMap<Integer,Integer> map = new HashMap<>();

        for(int ele : arr)
        {
            if(map.containsKey(ele))
                map.put(ele,map.get(ele)+1);
            else
                map.put(ele,1);
        }

        for(int key : map.keySet())
        {
            if(map.get(key) > n/2)
                return key;
        }

        return -1;
    }
}
```

Subarray with given sum

```
import java.util.HashMap;

public class Subarraysum {
    public static void main(String[] args) {

        int a[] = {10,15,-5,15,-10,5};
        int sum = 5;

        HashMap<Integer,Integer> map = new HashMap<>();

        int currentSum = 0 ;
        int start=0;
        int end = -1;

        for(int i=0 ; i<a.length ;i++)
        {
            currentSum += a[i];

            if(currentSum == sum)
            {
                start =0 ;
                end = i;
                break;
            }

            if( map.containsKey(currentSum-sum))
            {
                start = map.get(currentSum-sum) +1;
                end = i ;
                break;
            }
            map.put(currentSum, i);
        }

        if(end== -1)
            System.out.println("not found");
        else
            System.out.println(start+" "+end);
    }
}
```

49. Group Anagrams

Medium

🔖 Topics

🏢 Companies

Given an array of strings `strs`, group the **anagrams** together. You can return the answer in **any order**.

Example 1:

Input: `strs = ["eat","tea","tan","ate","nat","bat"]`

Output: `[["bat"],["nat","tan"],["ate","eat","tea"]]`

```
import java.util.*;

public class GroupAnagram {

    public static List<List<String>> groupAnagrams(String[] strs) {

        HashMap<String , List> map = new HashMap<>();

        for(String word : strs)
        {
            char arr[] = word.toCharArray();
            Arrays.sort(arr);

            String key = new String(arr);

            if(!map.containsKey(key))
            {
                map.put(key, new ArrayList<>());
            }
            map.get(key).add(word);
        }

        return new ArrayList<>(map.values());
    }

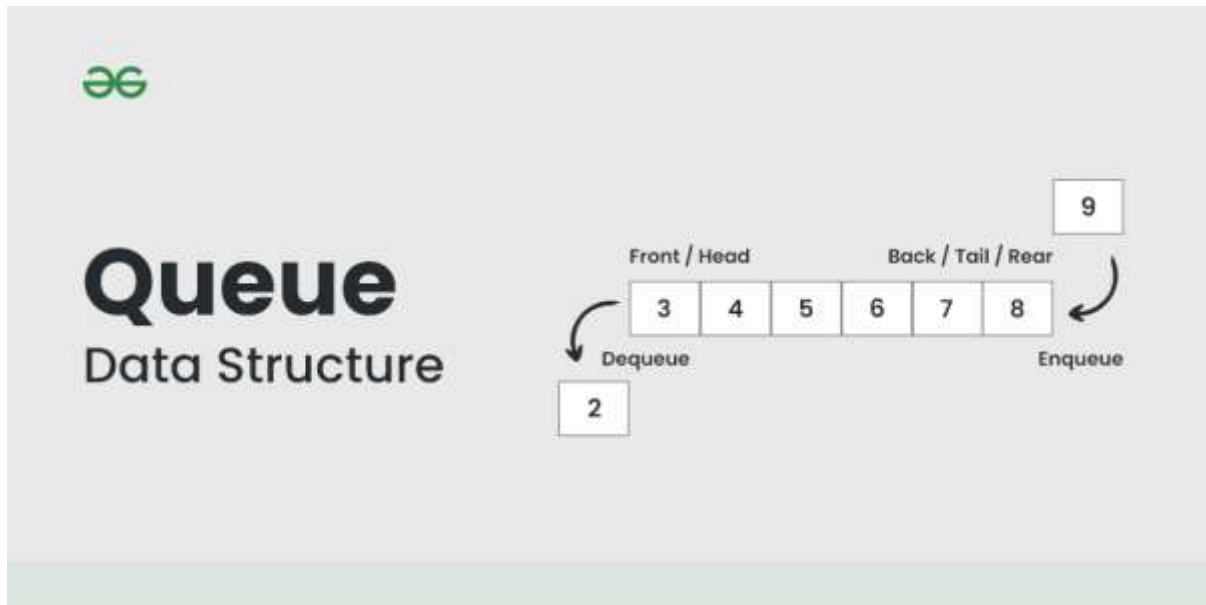
    public static void main(String[] args) {
        String a[] ={"eat","tea","tan","ate","nat","bat"};

        System.out.println(groupAnagrams(a));
    }
}
```

Queue

Queue is a data structure that follows => **FIFO (First In First Out)**

Insert → Rear Delete → Front



Real Life Example

- Ticket counter
- Line in ATM
- Printer queue

Queue Interface

Part of java.util package

It is an **interface**

Classes that Implement Queue

- LinkedList
- PriorityQueue
- ArrayDeque

Declaration

```
Queue<Integer> q = new LinkedList<>();
```

Important Points

- Queue is **FIFO**
- LinkedList is most commonly used
- PriorityQueue is **not FIFO**
- Allows duplicates

Queue Methods

Method	Return Type	Description
add(E e)	boolean	Adds element (throws error if full)
offer(E e)	boolean	Adds element (safe method)
remove()	E	Removes element (throws error if empty)
poll()	E	Removes element (returns null if empty)
element()	E	Returns front element (error if empty)
peek()	E	Returns front element (null if empty)

Difference

Method Pair	Difference
add() vs offer()	add → exception, offer → safe
remove() vs poll()	remove → exception, poll → null
element() vs peek()	element → exception, peek → null

PriorityQueue

Elements are stored in **sorted order**

Smallest element comes first

```
PriorityQueue<Integer> pq = new PriorityQueue<>();

pq.add(30);
pq.add(10);
pq.add(20);

System.out.println(pq); // [10, 30, 20]
```

```
import java.util.*;

class Demo {
    public static void main(String[] args) {

        Queue<Integer> q = new LinkedList<>();

        q.add(10);
        q.add(20);
        q.add(30);

        System.out.println(q); // [10, 20, 30]

        q.poll(); // removes 10

        System.out.println(q); // [20, 30]

        System.out.println(q.peek()); // 20
    }
}
```

215. Kth Largest Element in an Array

Medium

Topics

Companies

Given an integer array `nums` and an integer `k`, return the k^{th} largest element in the array.

Note that it is the k^{th} largest element in the sorted order, not the k^{th} distinct element.

Can you solve it without sorting?

Example 1:

Input: `nums = [3,2,1,5,6,4]`, `k = 2`

Output: 5

Example 2:

Input: `nums = [3,2,3,1,2,4,5,5,6]`, `k = 4`

Output: 4

```
import java.util.PriorityQueue;

public class KthLargest {
    public static void main(String[] args) {
        int a[] = {3,2,3,1,2,4,5,5,6};
        int k = 4;

        PriorityQueue<Integer> pq = new PriorityQueue<>();

        for(int i=0 ; i<k;i++)
        {
            pq.add(a[i]);
        }

        for(int i=k;i<a.length;i++)
        {
            if(a[i] > pq.peek())
            {
                pq.poll();
                pq.add(a[i]);
            }
        }

        System.out.println(pq.peek());
    }
}
```

Kth Smallest



Difficulty: **Medium**

Accuracy: **35.17%**

Submissions: **749K+**

Points: **4**

Average Time: **25m**

Given an integer array **arr[]** and an integer **k**, your task is to find and return the **kth smallest** element in the given array.

Note: The kth smallest element is determined based on the sorted order of the array.

Examples :

Input: arr[] = [10, 5, 4, 3, 48, 6, 2, 33, 53, 10], k = 4

Output: 5

Explanation: 4th smallest element in the given array is 5.

Input: arr[] = [7, 10, 4, 3, 20, 15], k = 3

Output: 7

Explanation: 3rd smallest element in the given array is 7.

```
import java.util.Collections;
import java.util.PriorityQueue;

public class KthSmallest {
    public static void main(String[] args) {
        int a[] = {3,2,3,1,2,4,5,5,6};
        int k = 4;

        PriorityQueue<Integer> pq = new
PriorityQueue<>(Collections.reverseOrder());

        for(int i=0 ; i<k;i++)
        {
            pq.add(a[i]);
        }

        for(int i=k;i<a.length;i++)
        {
            if(a[i] < pq.peek())
            {
                pq.poll();
                pq.add(a[i]);
            }
        }
        System.out.println(pq.peek());
    }
}
```